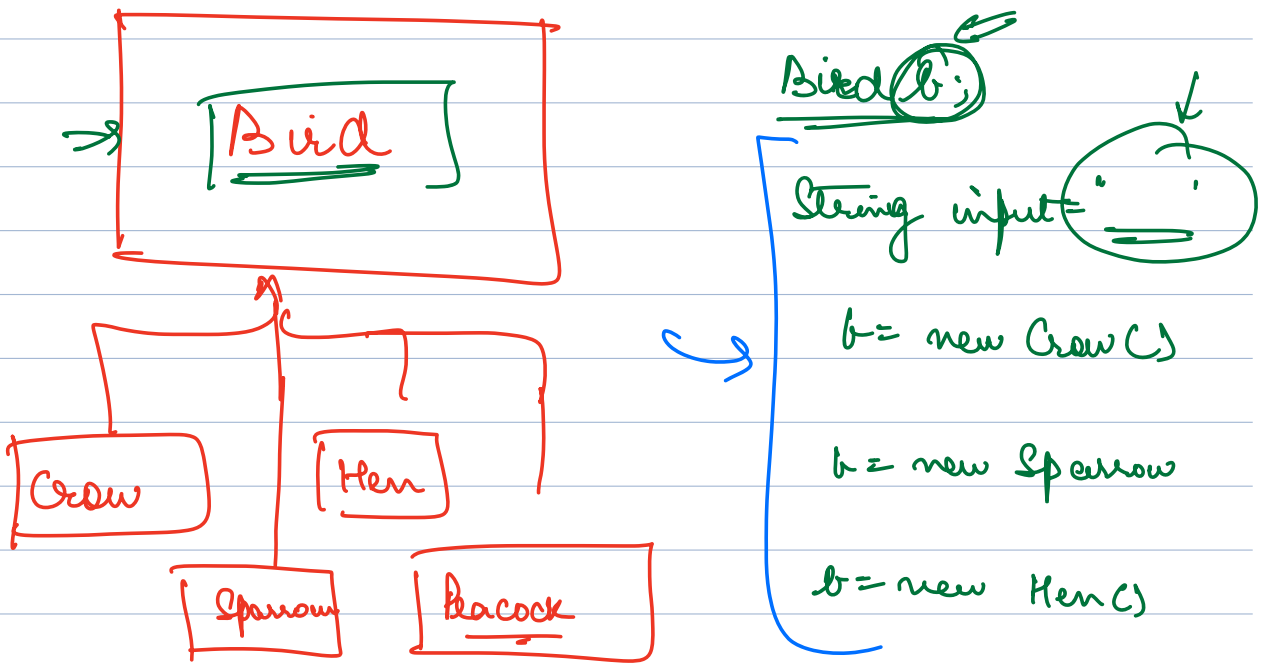
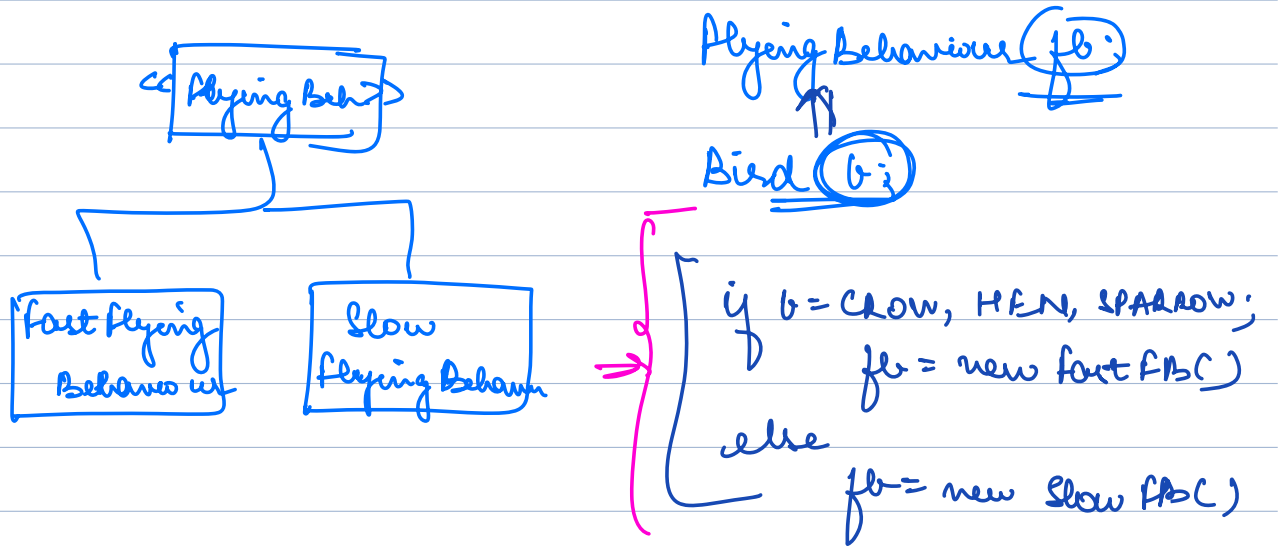
Agenda ⓪ Practical Factory Design Pattern
① Factory Method Design Pattern
② Abstract Factory Design Pattern

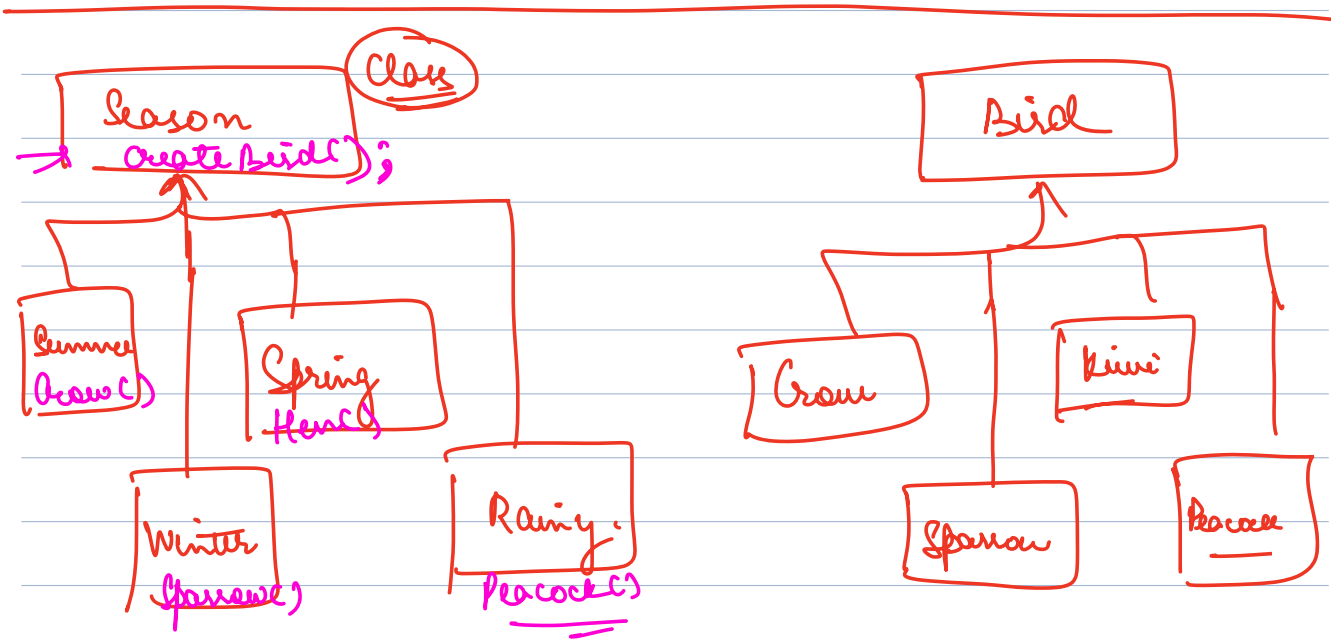
Practical Factory Design Pattern

→ Whenever we have to create obj of a specific type of particular class/interface based on some input.





⇒ It moves the responsibility of creating obj. of corresponding class from Client to Factory Class



Bird factory {

Create Happier Bird of Season (season) {

if (season. equals (SUMMER)) {

}

}

Season {

create Bird() {

⇒ return Crow();

}

}

Practical factory

Client {

⇒ Season s ;

Bird b = ;

//b = BirdFactory.getBirdOfSeason(s);

Bird b = S.CreateBird();

}

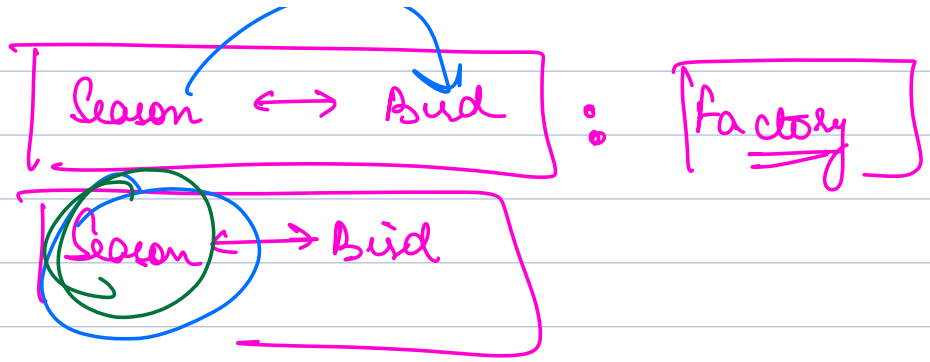
~~Client~~

No ~~central~~ place

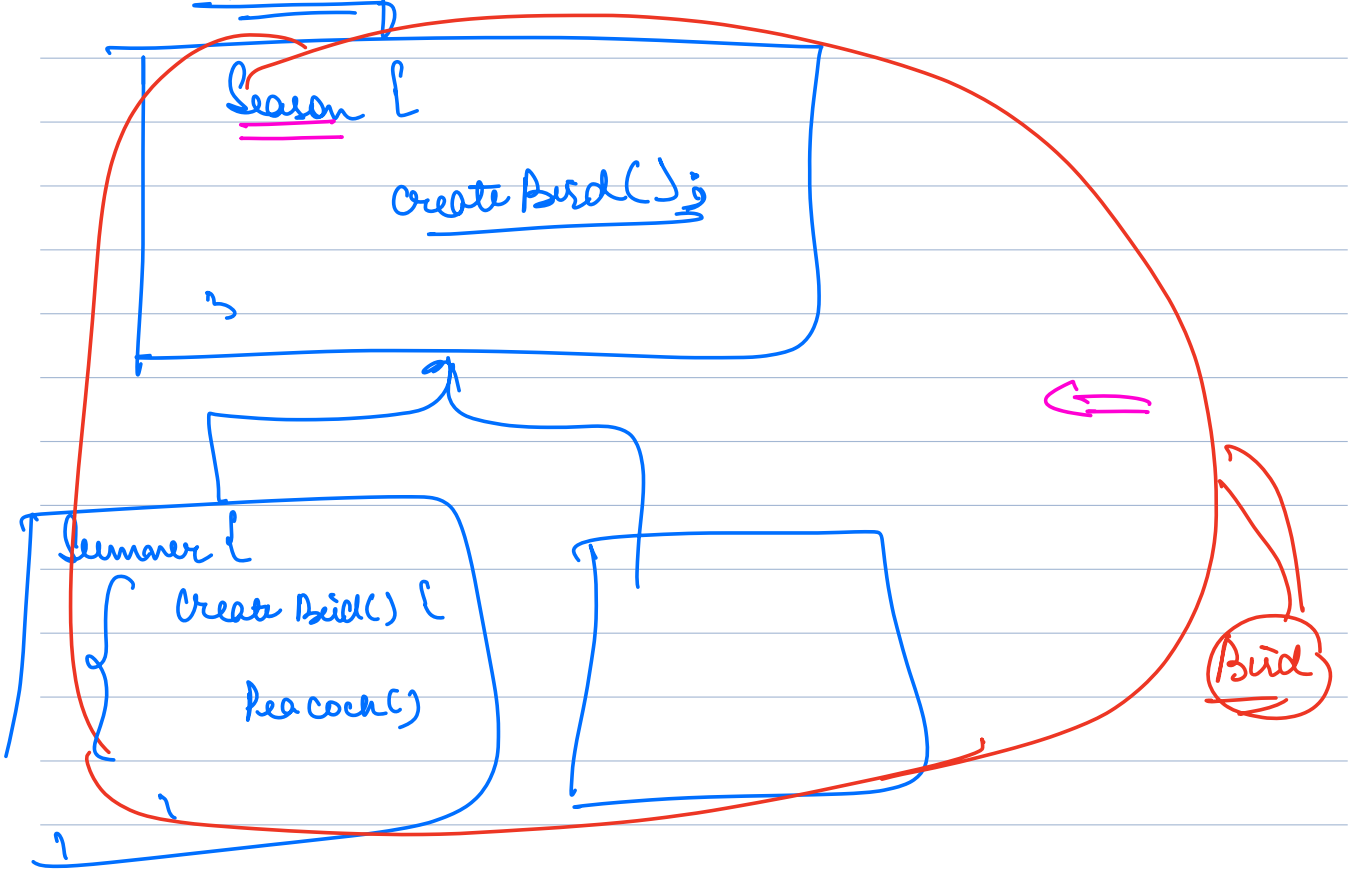
Class

put method there

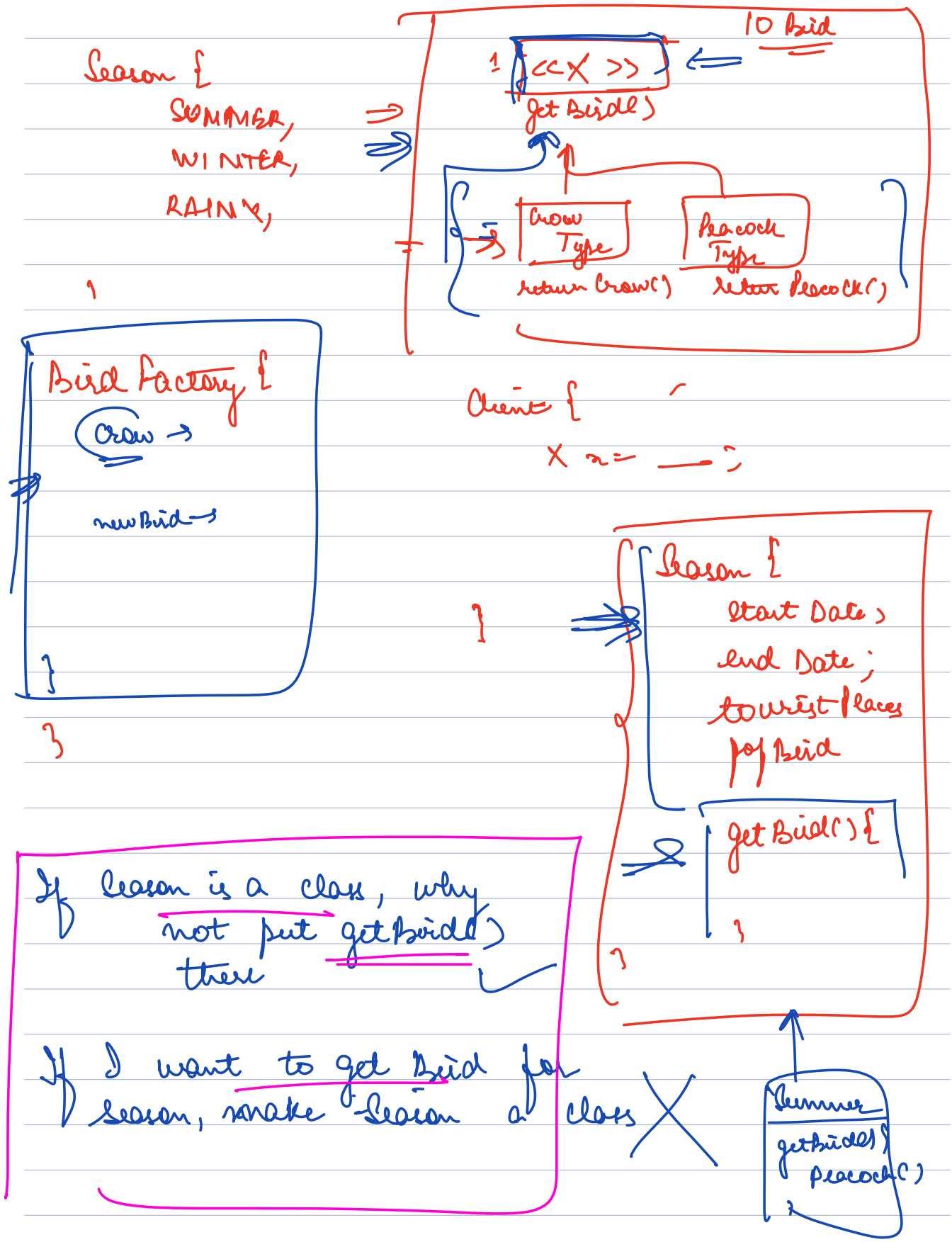
anything else, Enum ⇒ use factory

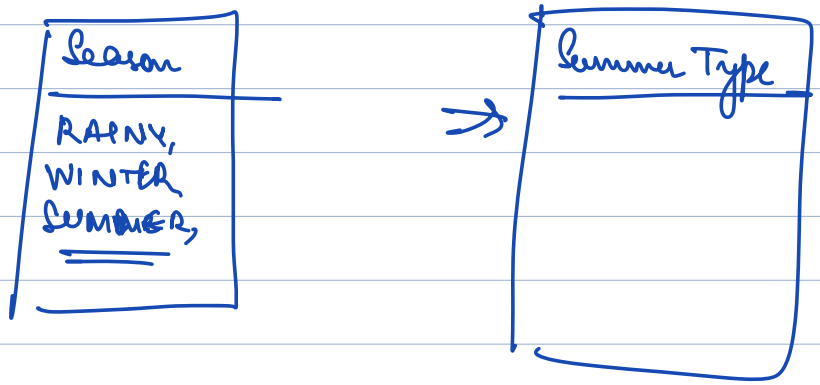


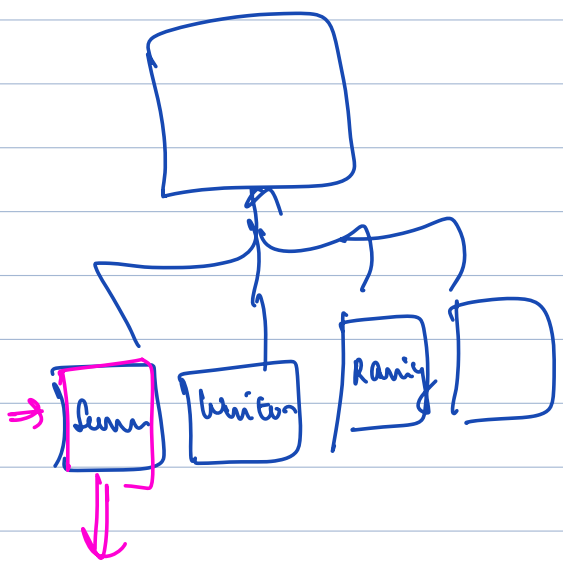
Why do I need a separate class to get corn bird - why can't that logic be in season class itself?



Pro: Difficult to miss ⇒ less Prone to error


Season {
SUMMER,
WINTER,
RAINY,
}
10 Bird
1
<<X>>
getBird()
Crow Type
Peacock Type
return Crow()
return Peacock()
Bird Factory {
Crow →
newBird →
}
Client {
X x = —;
}
Season {
start Date;
end Date;
touristPlaces
of Bird
getBird() {
}
}
If Season is a class, why not put getBird() there
If I want to get Bird for season, make Season a class
Summer
getBird()
peacock()






```
getBuid(){
    return Reason();
}
```

Switch (birdType) {

,

Case SPARROW:

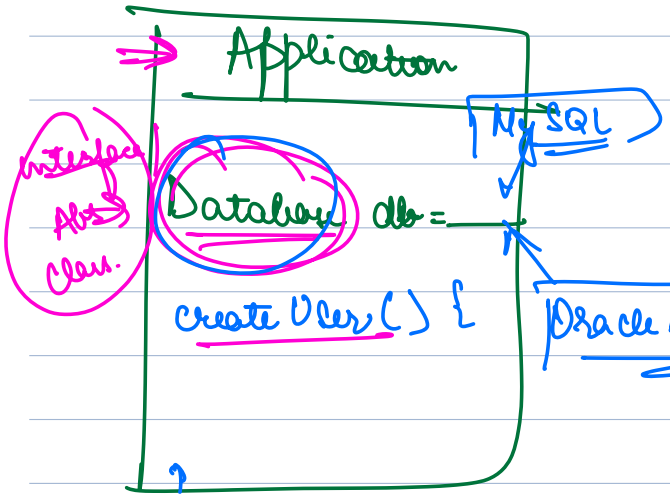
}

---

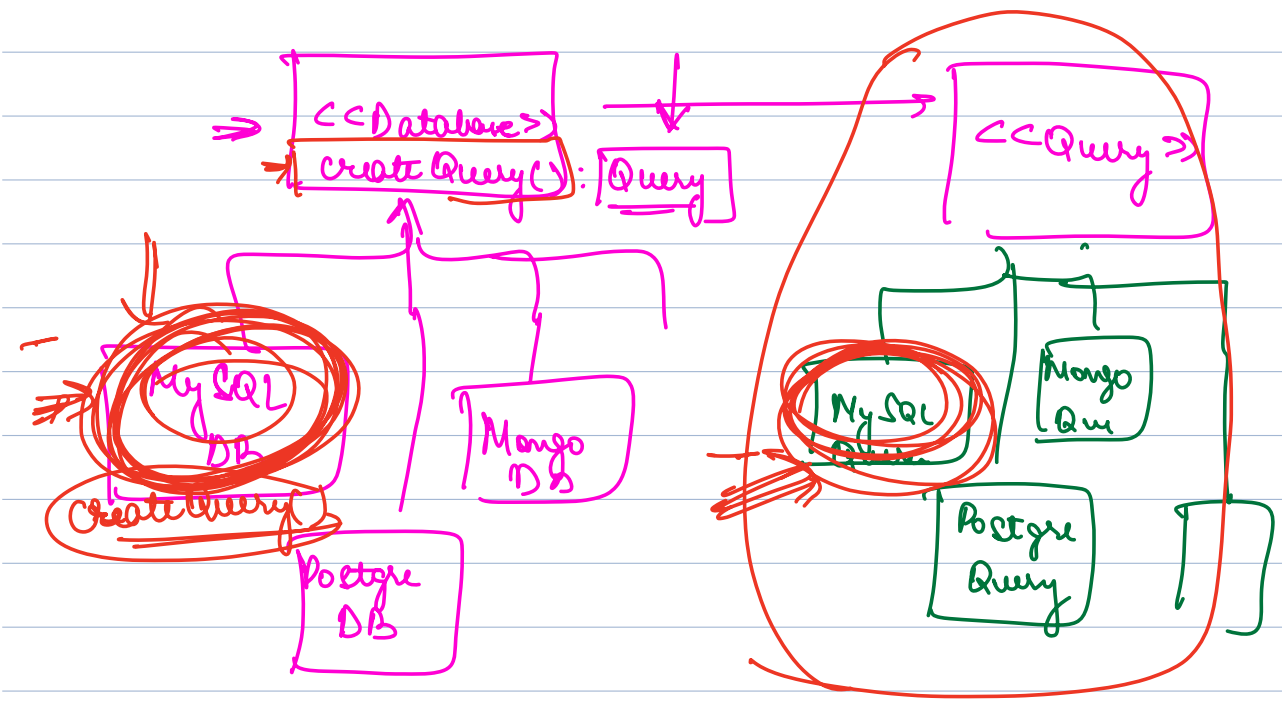
① Whenever based on cond^{n} we need to create an obj of a particular classes, put logic of cond^{n} → classes mapping in.

Factory

② If the cond^{n} is that the particular obj is of a particular class, why not have that mapping within that class itself.



Dependency Inversion :
No 2 concrete classes should directly talk to each other



Application {

Database db = ——————;

Query q;

Query Factory

Create Query for DB ( ) {

if db = —

else db = —

}

}

⇒ Often when implementing an interface / inheritance rel^n, I also create a relation to another inheritance you want to map

⇒ When a specific type of one class you need to a specific type of another class

MySQL DB

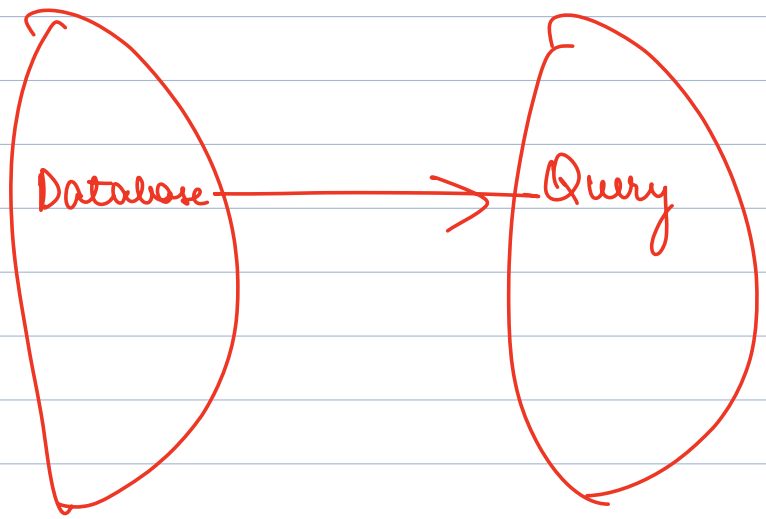
Create Query() {

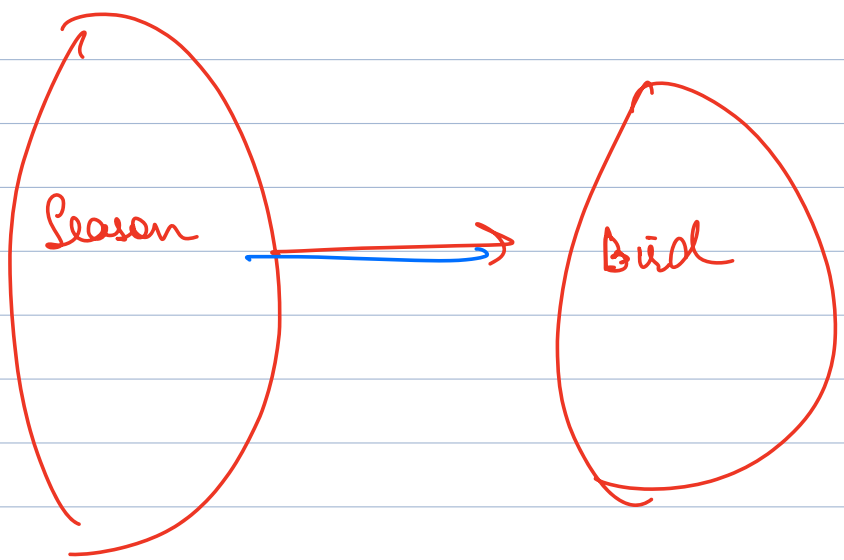
return MySQL Query();

}

Factory Method Design Pattern

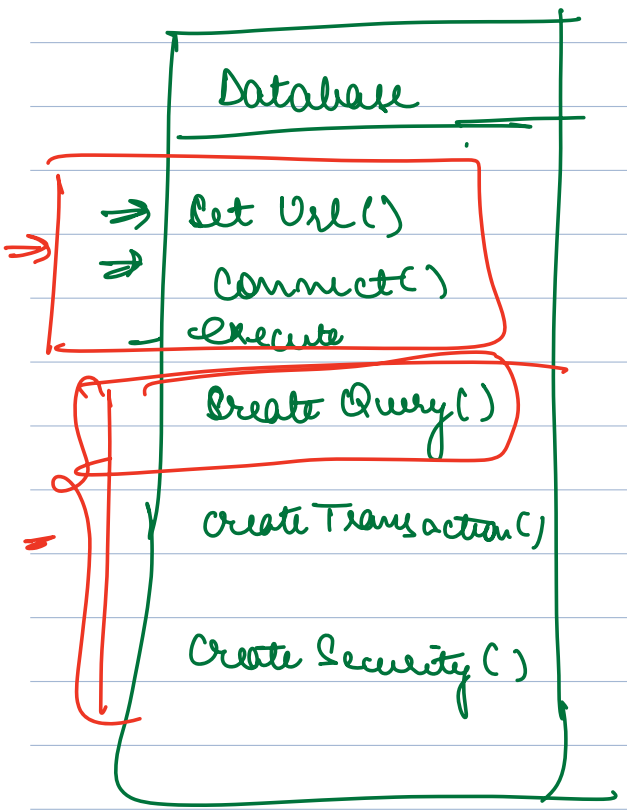
⇒ Allows to have mapping b/w corresponding classes in 2 inheritance.





Factory ① To create a new obj.
② Corresponding obj.

Existence of method whose only purpose is to create an object of corresponding class



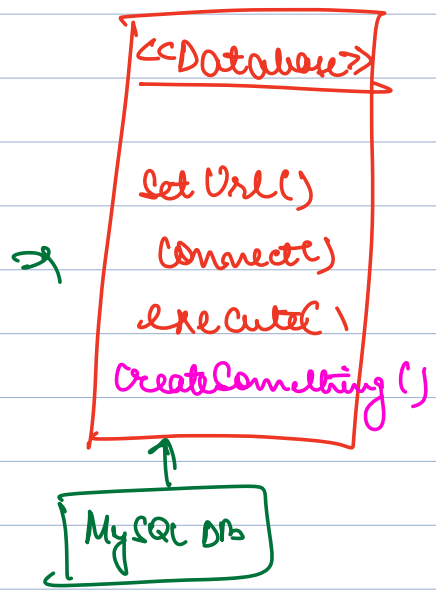
Problem

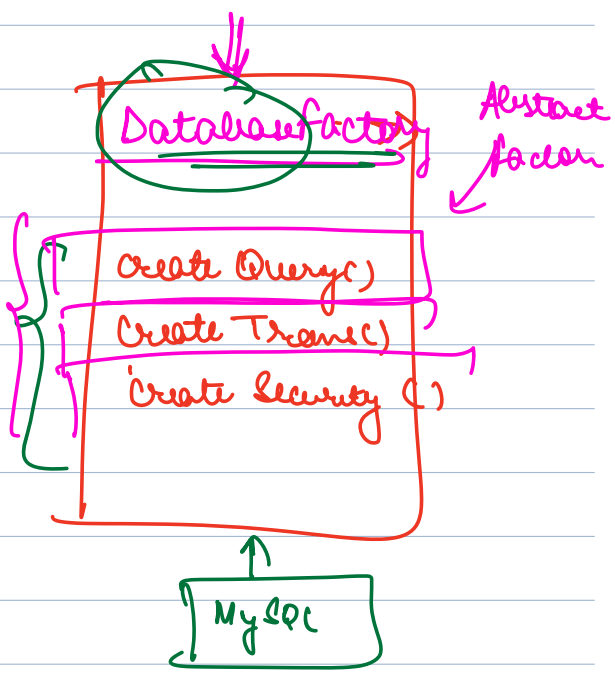
⇒ SRP is violated

① Methods a database must have

② Create objs of corresponding ~~type~~

SRP ⇒ Split



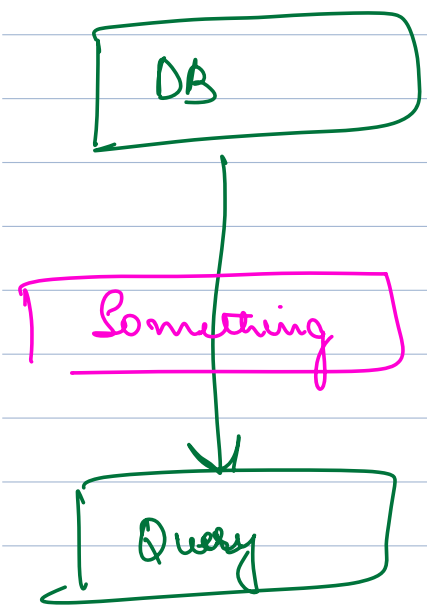


MySQLQuery MySQLTrans MySQLRecord

Client {

Database db = new MySQLDatabase();

Query q = db.getSomething().getQuery();

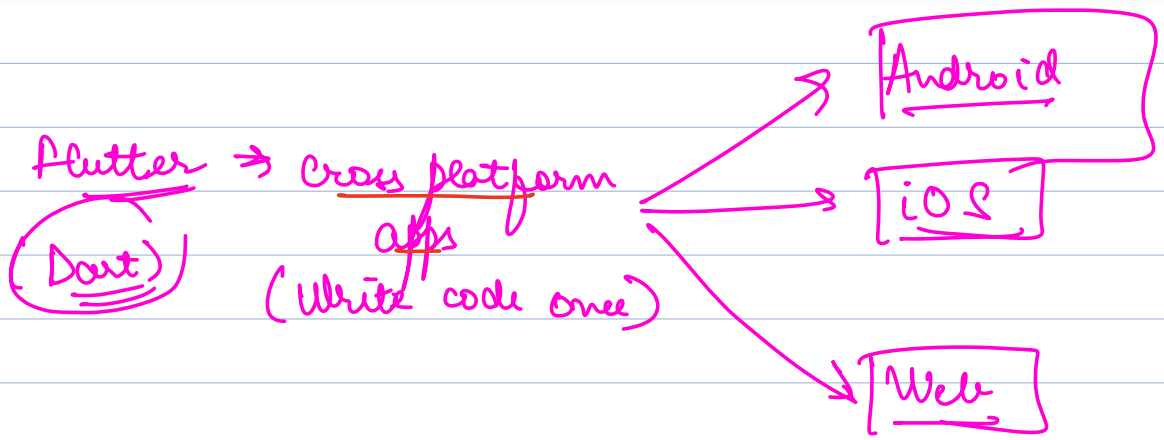


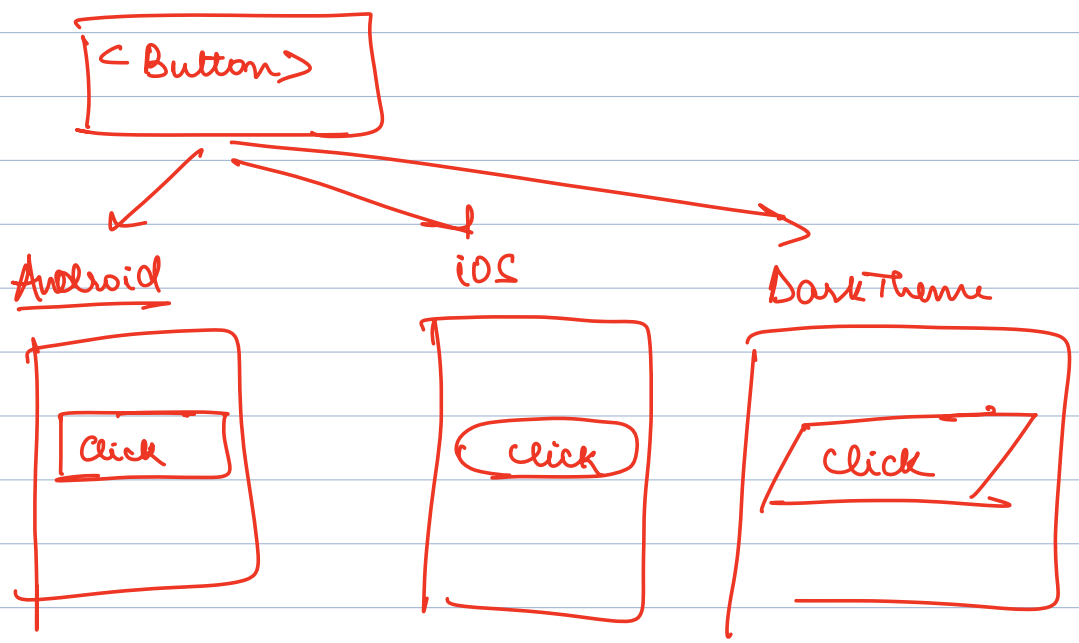
Abstract factory ⇒ extension of factory Method

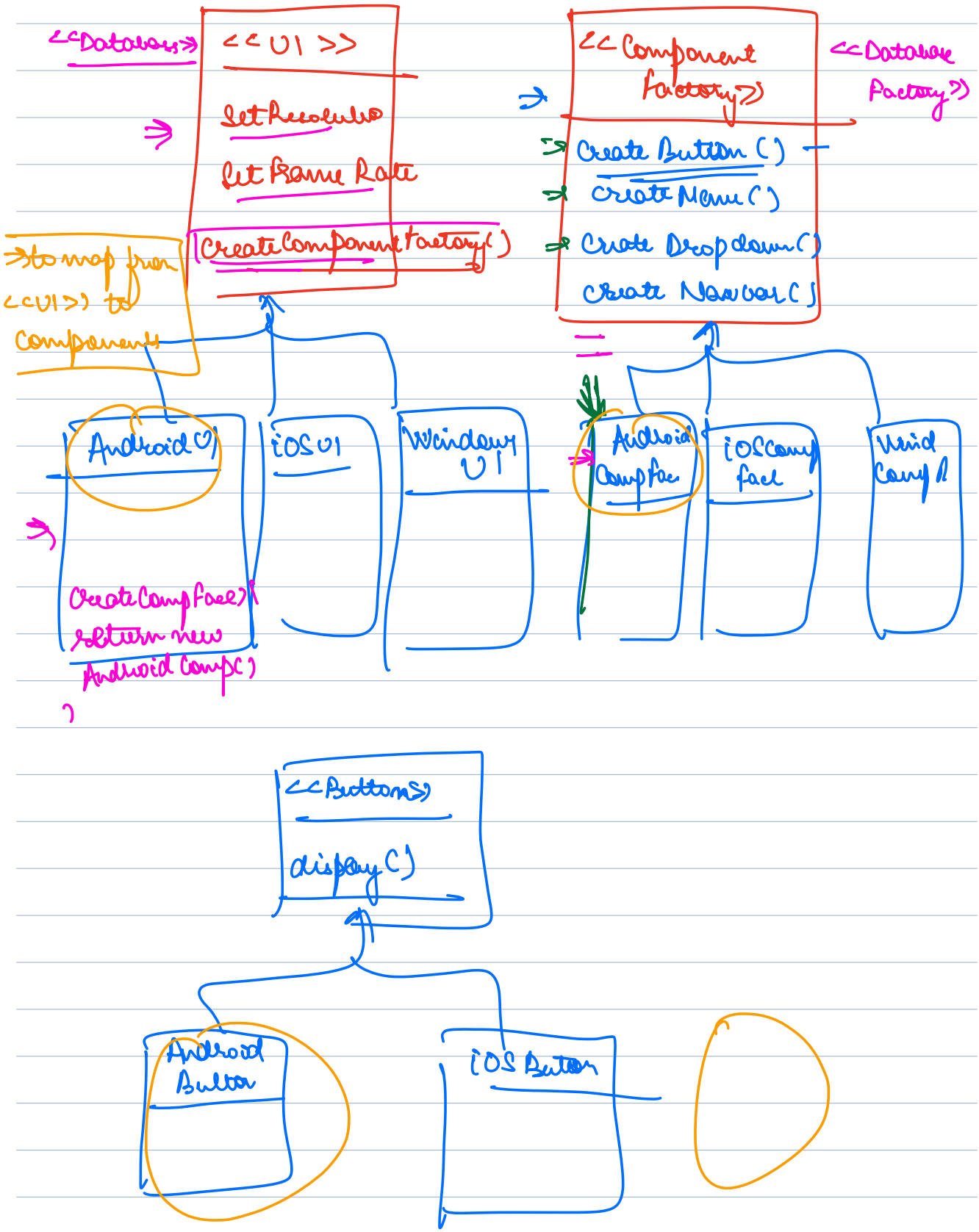
⇒ You extract out all of the factory methods from an interface to another interface

→ Because we want to follow SRP

→ In abs factory, return type of methods is of corresponding class.






<<Dataclass>>
<<UI>>
SetResolution
SetFrameRate
CreateComponentFactory()
to map from <<UI>> to Components
<<ComponentFactory>>
<<DatabaseFactory>>
CreateButton()
CreateMenu()
CreateDropdown()
CreateNavbar()
AndroidUI
iOSUI
WindowsUI
AndroidCompFac
iOSCompFac
WindCompFac
CreateCompFac()
{return new AndroidComp()}
}
<<Buttons>>
display()
AndroidButton
iOSButton

Practical Factory

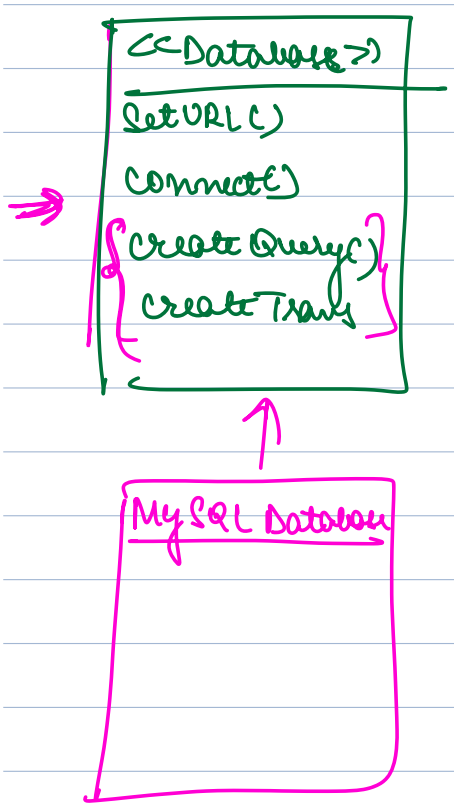
Create an obj of corresponding sub class based on a cond^n.

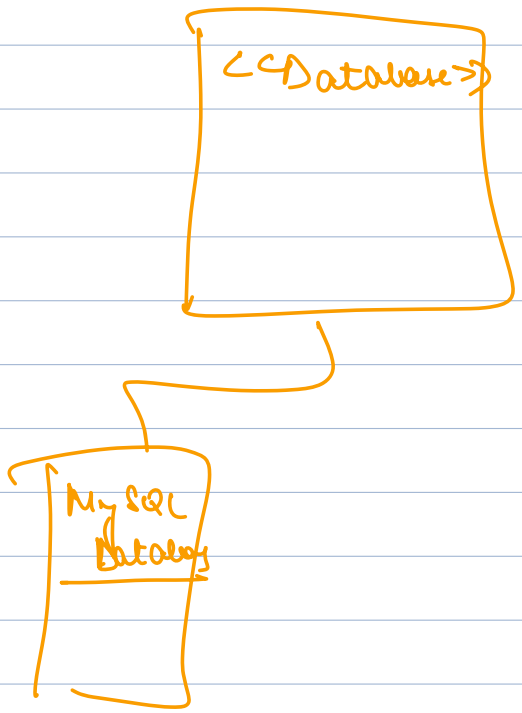
Factory Method

A method in an interface which returns a new obj of a corresponding interface.

Abstract Factory

An interface whose purpose corresponding obj of other interfaces





Config
dbUrl="

```
Database practical factory {
    get Database from Url() {
        if url.startsWith("mySQL://")
        return new MySQLDB()
    }
}
```

Adapter, Facade ⇒ Next Class

Break on Friday

Doubt solving session ⇒ Sunday

Next Steps

1. Wed: Easy DP: adapter, facade — Assgn Ques
2. Break on Friday
3. Reading material for all DP on Wed
4. T, F, S, Sunday Mor: Revise, Read, Code all DP
5. Sunday noon: Doubt session

Next Break around Case Study

TTTT → 1 day

BMC → 1 day